

Discussion sur les sections critiques

Fabrice.Kordon@lip6.fr



En guise d'introduction

2

 **Se poser la question de la protection des ressources...**

 Cela doit-il être systématique?



 Peut-on réfléchir un peu?

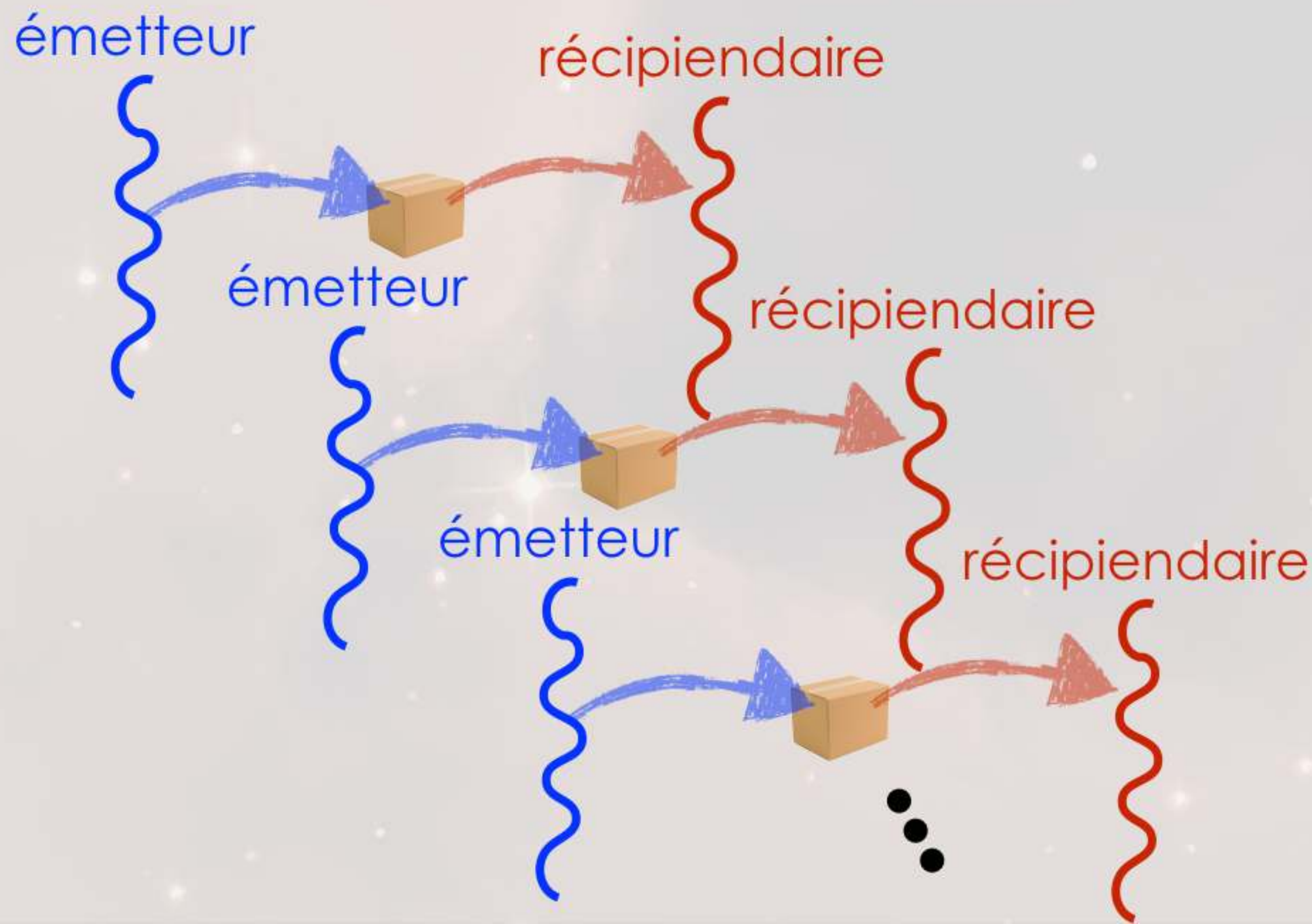
► **Via un exemple**

Groupes de producteurs/consommateurs

3

Chaque groupe partage une «boîte» dédiée

-  Une valeur
-  Un booléen indiquant que la valeur a été déposée

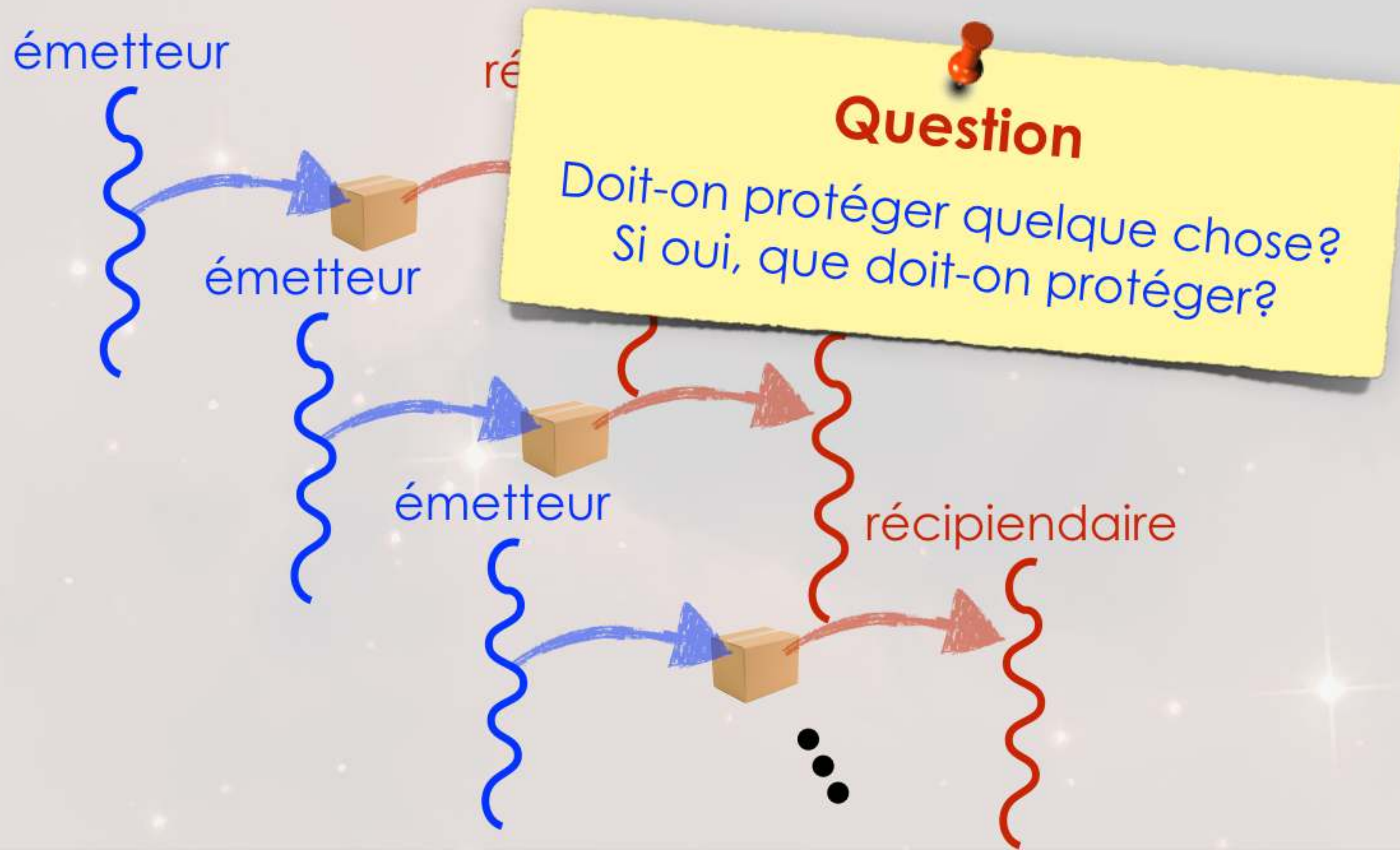


Groupes de producteurs/consommateurs

3

Chaque groupe partage une «boîte» dédiée

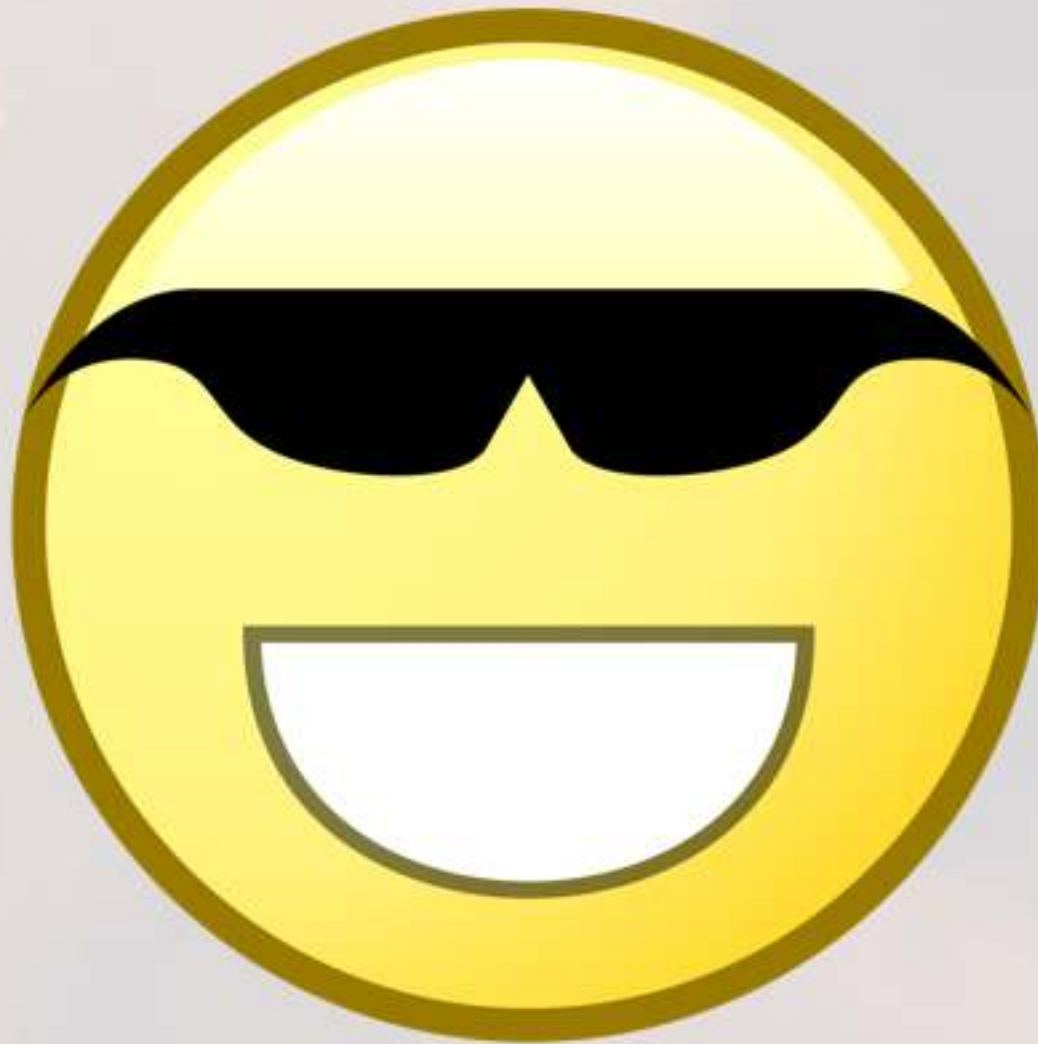
-  Une valeur
-  Un booléen indiquant que la valeur a été déposée



La classe BoiteMessageMinimaliste

4

```
class BoiteMessageMinimaliste {  
    public boolean plein;  
    public int valeur;  
  
    BoiteMessageMinimaliste() {  
        plein = false;  
    }  
}
```



La classe Recipiendaire

5

```
class Recipiendaire implements Runnable {
    private static int cpt = 1;
    private static final Object mutex = new Object();
    private int monNumero;
    private BoiteMessageMinimaliste b;
    Recipiendaire (BoiteMessageMinimaliste b) {
        synchronized(mutex) {
            monNumero = cpt++;
        }
        this.b = b;
    }
    private void trace (String m) {
        System.out.println ("      Recipiendaire " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }
}
```


La classe Recipiendaire

5

```
class Recipiendaire implements Runnable {
    private static int cpt = 1;
    private static final Object mutex = new Object();
    private int monNumero;
    private BoiteMessageMinimaliste b;
    Recipiendaire (BoiteMessageMinimaliste b) {
        synchronized(mutex) {
            monNumero = cpt++;
        }
        this.b = b;
    }
    private void trace (String m) {
        System.out.println ("      Recipiendaire " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        this.trace ("initialisé");
        while (! b.plein) {
            try {
                this.trace ("tente une lecture");
                Thread.sleep(1000); // boucle semi-active
            }
            catch (Exception e2) {
                this.trace(e2 + "Ne doit pas arriver");
            }
        }
        this.trace ("retire " + b.valeur);
        this.trace ("terminé");
    }
}
```


La classe Emetteur

6

```
class Emetteur implements Runnable {
    private static int cpt = 1;
    private static final Object mutex = new Object();

    private BoiteMessageMinimaliste b;
    private int monNumero;
    private boolean actif = true;

    Emetteur (BoiteMessageMinimaliste b) {
        synchronized(mutex) {
            monNumero = cpt++;
        }
        this.b = b;
    }

    private void trace (String m) {
        System.out.println ("Emetteur " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        this.trace ("initialisé");
        b.valeur = monNumero;
        b.plein = true;
        this.trace ("terminé");
    }
}
```


La classe principale

7

```
public class GroupeCommunicant {  
  
    static final public int cstMaxGroupe = 3; // constante  
  
    public static void main(String []args) {  
  
        BoiteMessageMinimaliste boite;  
        Thread recipiendaires[] = new Thread [cstMaxGroupe];  
        Thread emetteurs[] = new Thread [cstMaxGroupe];  
  
        // démarrer les groupes  
        for (int i = 0; i < cstMaxGroupe ; i++) {  
            // La boite dédiée  
            boite = new BoiteMessageMinimaliste ();  
            // Le Recipiendaire (qui reçoit)  
            recipiendaires[i] = new Thread(new Recipiendaire (boite));  
            recipiendaires[i].start();  
            // Le Emetteur (qui émet)  
            emetteurs[i] = new Thread(new Emetteur (boite));  
            emetteurs[i].start();  
        }  
    }  
}
```


La classe principale

7

```
public class GroupeCommunicant {
```

```
$ java GroupeCommunicant
```

```
    Recipiendaire 1 : initialisé
```

```
Emetteur 1 : initialisé
```

```
    Recipiendaire 2 : initialisé
```

```
Emetteur 1 : terminé
```

```
Emetteur 2 : initialisé
```

```
    Recipiendaire 1 : tente une lecture
```

```
    Recipiendaire 3 : initialisé
```

```
Emetteur 2 : terminé
```

```
    Recipiendaire 2 : tente une lecture
```

```
    Recipiendaire 3 : tente une lecture
```

```
Emetteur 3 : initialisé
```

```
Emetteur 3 : terminé
```

```
    Recipiendaire 2 : retire 2
```

```
    Recipiendaire 1 : retire 1
```

```
    Recipiendaire 3 : retire 3
```

```
    Recipiendaire 1 : terminé
```

```
    Recipiendaire 2 : terminé
```

```
    Recipiendaire 3 : terminé
```


La classe principale

7

```
public class GroupeCommunicant {
```

```
$ java GroupeCommunicant
```

```
    Recipiendaire 1 : initialisé
```

```
    Recipiendaire 1 : tente une lecture
```

```
Emetteur 1 : initialisé
```

```
    Recipiendaire 2 : initialisé
```

```
Emetteur 1 : terminé
```

```
Emetteur 2 : initialisé
```

```
    Recipiendaire 2 : tente une lecture
```

```
Emetteur 2 : terminé
```

```
Emetteur 3 : initialisé
```

```
    Recipiendaire 3 : initialisé
```

```
Emetteur 3 : terminé
```

```
    Recipiendaire 3 : retire 3
```

```
    Recipiendaire 3 : terminé
```

```
    Recipiendaire 1 : retire 1
```

```
    Recipiendaire 2 : retire 2
```

```
    Recipiendaire 1 : terminé
```

```
    Recipiendaire 2 : terminé
```


En guise de conclusion...

8

Ici, pas besoin d'avoir de protection...

-  On pouvait y penser quand même...

Alors quand protéger?

-  Synchroniser les «usagers»
-  Ces mécanismes ont un coût
 - ▶ Éviter de le faire inutilement?
 - ▶ Attention à tracer les cas délicats (évolution du logiciel)

Pour votre culture, $\exists$ classes d'algorithmes

-  Avec verrous
 -  Sans verrou (lock-free)
 -  Sans attente (wait-free)
- Hypothèses de fonctionnement
- Opérations atomiques
(TAS & CAS)

